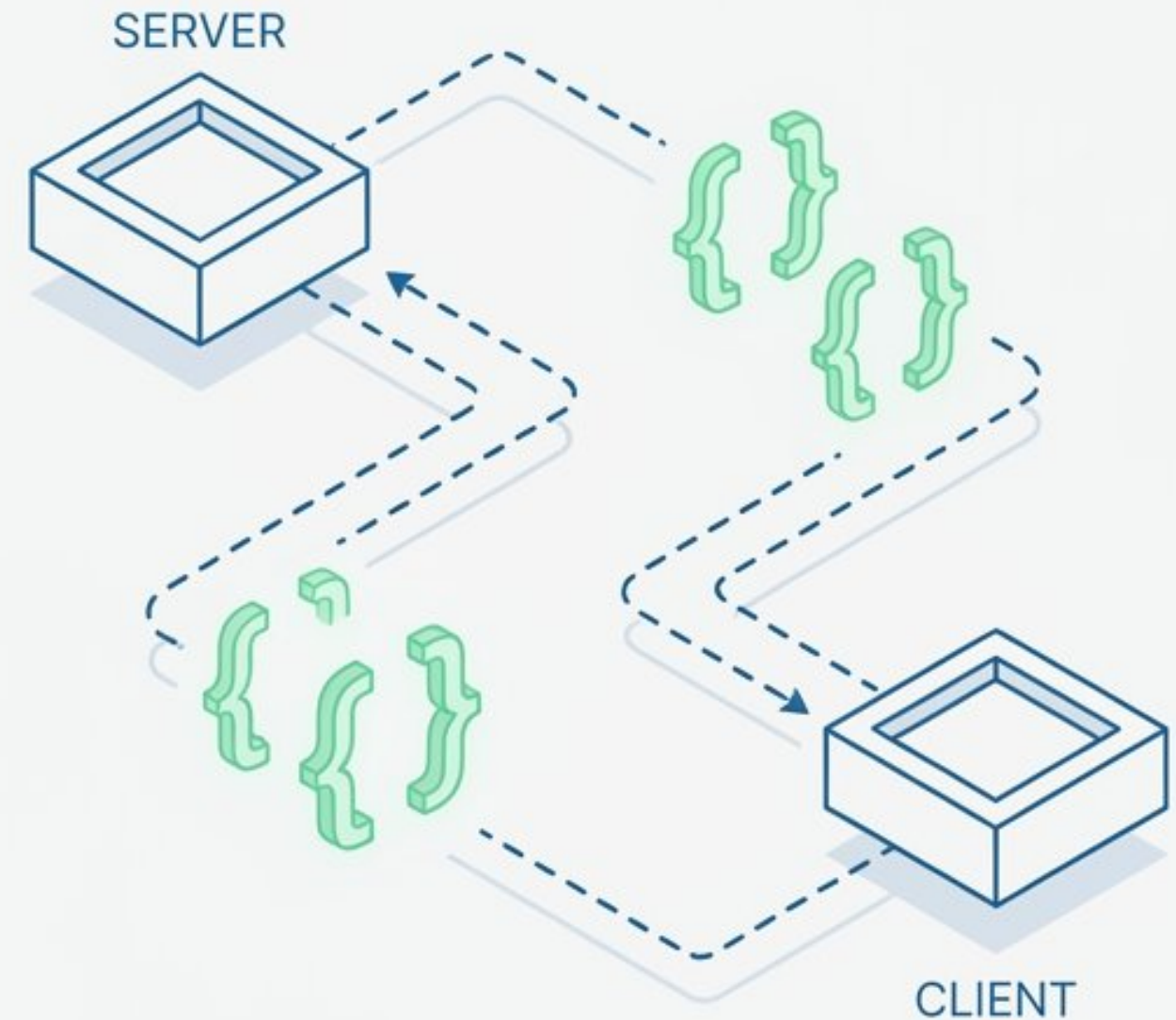


TEC0748 – Programação Web

Semana 10: JSON na Prática e Requisições AJAX

O guia definitivo para comunicação assíncrona e troca de dados estruturados no navegador.



Mapa de Engenharia: Objetivos da Semana

1



Manipular dados em formato JSON nativamente no cliente (Parse & Stringify).

2



Mapear a infraestrutura da web compreendendo APIs e Endpoints.

3



Orquestrar requisições HTTP assíncronas utilizando a moderna Fetch API.

4



Dinamizar interfaces atualizando o DOM com dados externos sem recarregar a página.

O Padrão Ouro da Troca de Dados: JSON



JavaScript Object Notation.



Formato leve e de texto puro (independente de linguagem).



Regra de Ouro: Aspas duplas obrigatórias nas chaves e strings.

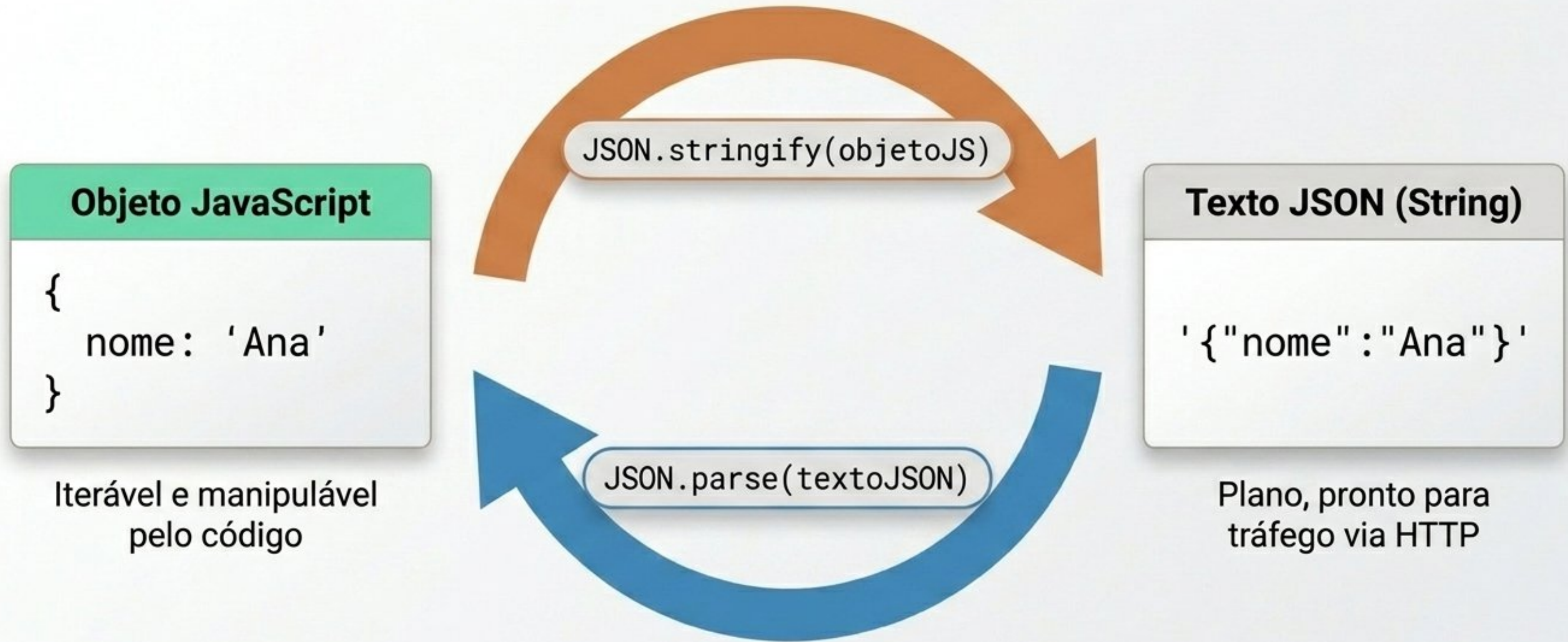
```
{  
  "id": 1,  
  "nome": "Ana",  
  "ativo": true  
}
```

Valor numérico (sem aspas)

Chave obrigatória em aspas duplas

Booleano nativo

O Tradutor de Redes: Parse vs. Stringify



Dissecando um JSON Aninhado (Nested JSON)

```
let estudante = {  
  "id": 101,  
  "notas": [7.5, 9.0],  
  "endereco": {  
    "cidade": "São Paulo"  
  }  
};
```

estudante.id

Retorna: 101

estudante.notas[0]

Retorna: 7.5 (Array começa em 0)

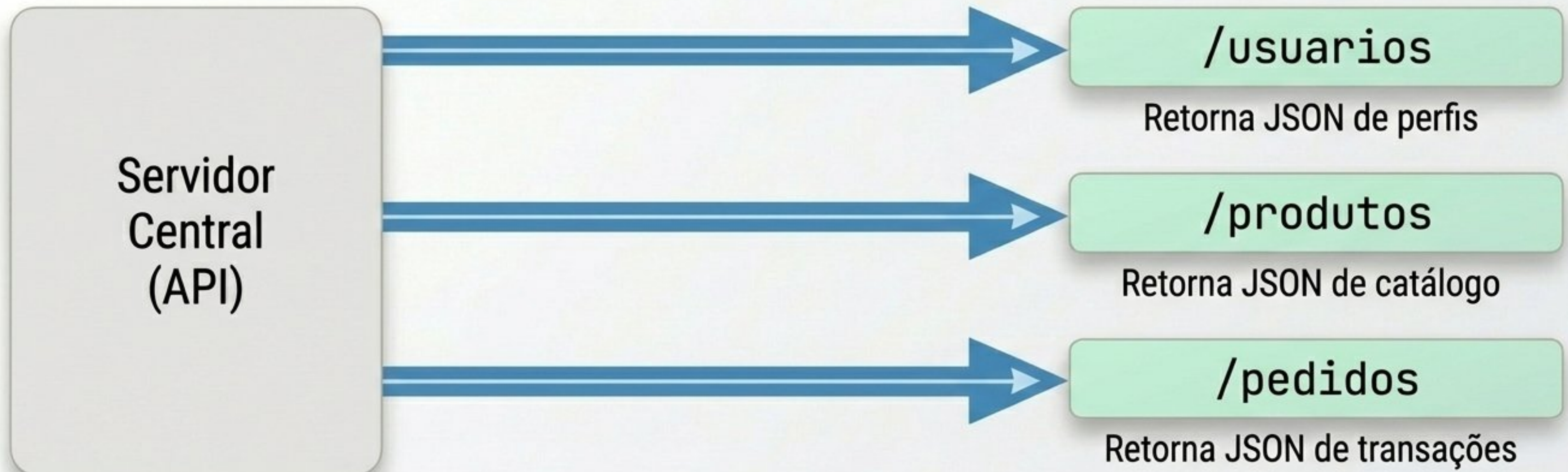
estudante.endereco.cidade

Retorna: 'São Paulo'

A Infraestrutura Externa: APIs e Endpoints

API Web: O catálogo de serviços de um servidor (ex: 'O Restaurante').

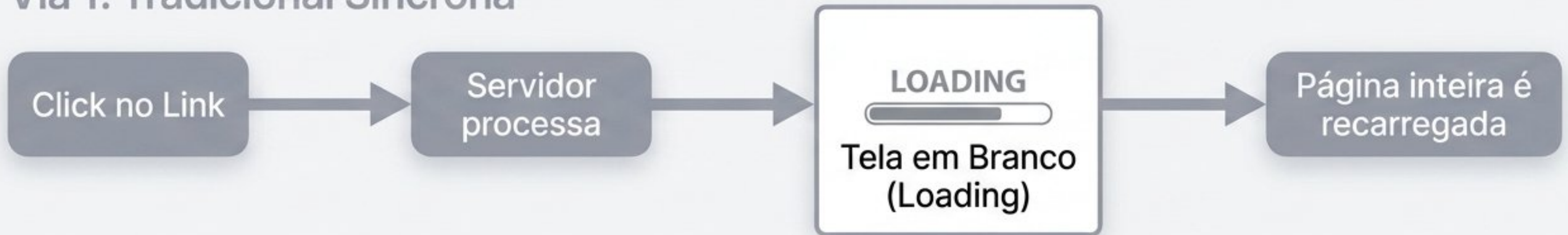
Endpoint: A URL específica que retorna um recurso exato (ex: 'O Prato do Menu').



O Motor AJAX: Quebrando a Sincronia

Atualiza apenas fragmentos da tela sem congelar o navegador.

Via 1: Tradicional Síncrona



Via 2: AJAX Moderna



O Veículo Moderno: A Fetch API e as Promises

```
fetch('https://api.exemplo.com/dados')
```

Substitui o antigo XMLHttpRequest com uma sintaxe limpa.

A Metáfora da Promise

Uma chamada `fetch()` não devolve o dado na hora. Ela devolve uma Promise.



Você recebe um ticket. Não precisa ficar parado esperando. Você continua navegando (assincronicidade) e, quando o pacote JSON fica pronto, você é avisado para agir.

Anatomia de uma Requisição Fetch

```
fetch('https://api.test/users')
```

O Disparo: Vai à rede e retorna uma Promise bruta do HTTP.

```
.then(resposta => resposta.json())
```

A Tradução: Intercepta a resposta e faz o 'parse' automático do JSON.

```
.then(dados => console.log(dados))
```

O Destino Final: Captura o Objeto JS manipulável para injetar na aplicação.

```
.catch(erro => console.error(erro))
```

A Rede de Segurança: Captura e avisa sobre links quebrados ou redes offline.

Síntese de Arquitetura: O Flowchart Cliente-Servidor



Roteiro do Laboratório Prático



Etapa 1: O Setup Seguro

Criar um objeto JSON fixo direto no script como constante para validar a exibição inicial.



Etapa 2: Injeção no DOM

Percorrer este JSON com laço 'for' e formatar como elementos dinâmicos em uma HTML.



Etapa 3: Chamada Externa

Substituir a variável local por um fetch() chamando um endpoint de teste da web.



Etapa 4: Interface Reativa

Adicionar um botão 'Carregar' que dispara a requisição sem recarregar a aba do navegador.

Atividade Verificadora e Critérios de Sucesso

Requisitos Mínimos

- ✓ [OK] Realizar a leitura assíncrona de um arquivo ou endpoint JSON.
- ✓ [OK] Extrair dados de um array aninhado sem erros de índice.
- ✓ [OK] Renderizar dinamicamente pelo menos 2 campos na interface HTML (ex: Nome e E-mail).

Atenção de Segurança

- ! [!] Tratamento de falhas: incluir estrutura `.catch()` obrigatória para quedas de rede.
- ! [!] Evitar seleções incorretas de nós inativos ou inexistentes no DOM.

O Próximo Nível: Dando Vida ao Backend (Semana 11)



O Client-Side está completo. O motor de envio e recebimento funciona.
Na próxima semana, você aprenderá a criar seus próprios Endpoints dinâmicos com PHP.
O sistema passará a ser uma aplicação completa baseada em arquitetura de dados reais.